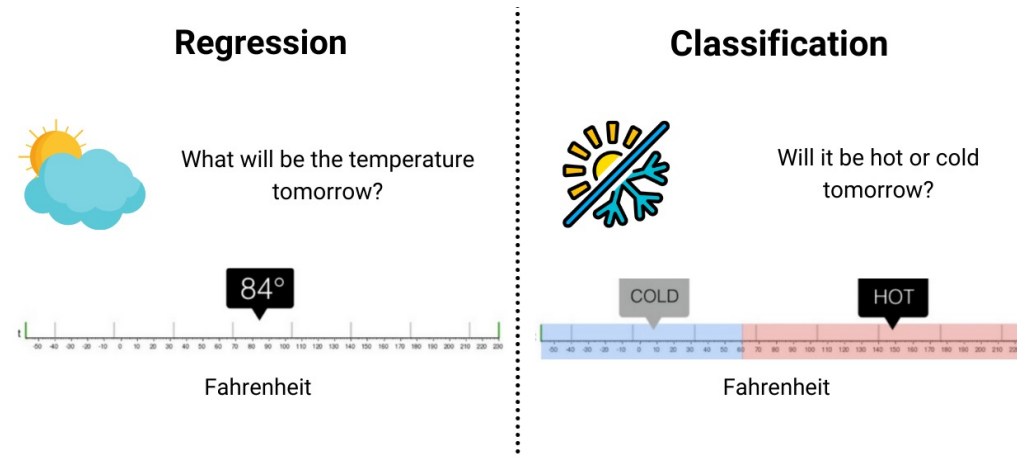


Types of Learning

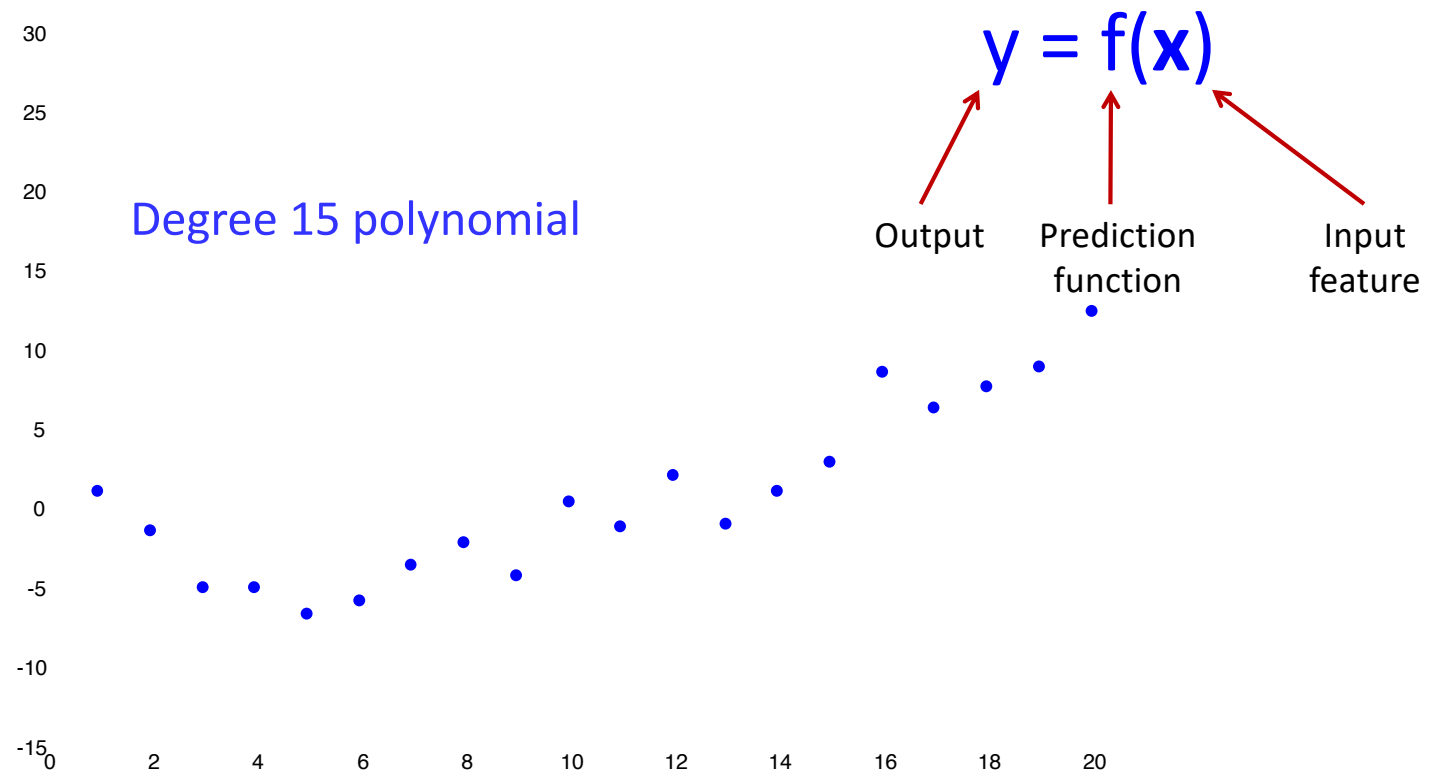
	<i>Supervised Learning</i>	<i>Unsupervised Learning</i>
<i>Discrete</i>	classification or categorization	clustering
<i>Continuous</i>	regression	dimensionality reduction

Regression and Classification

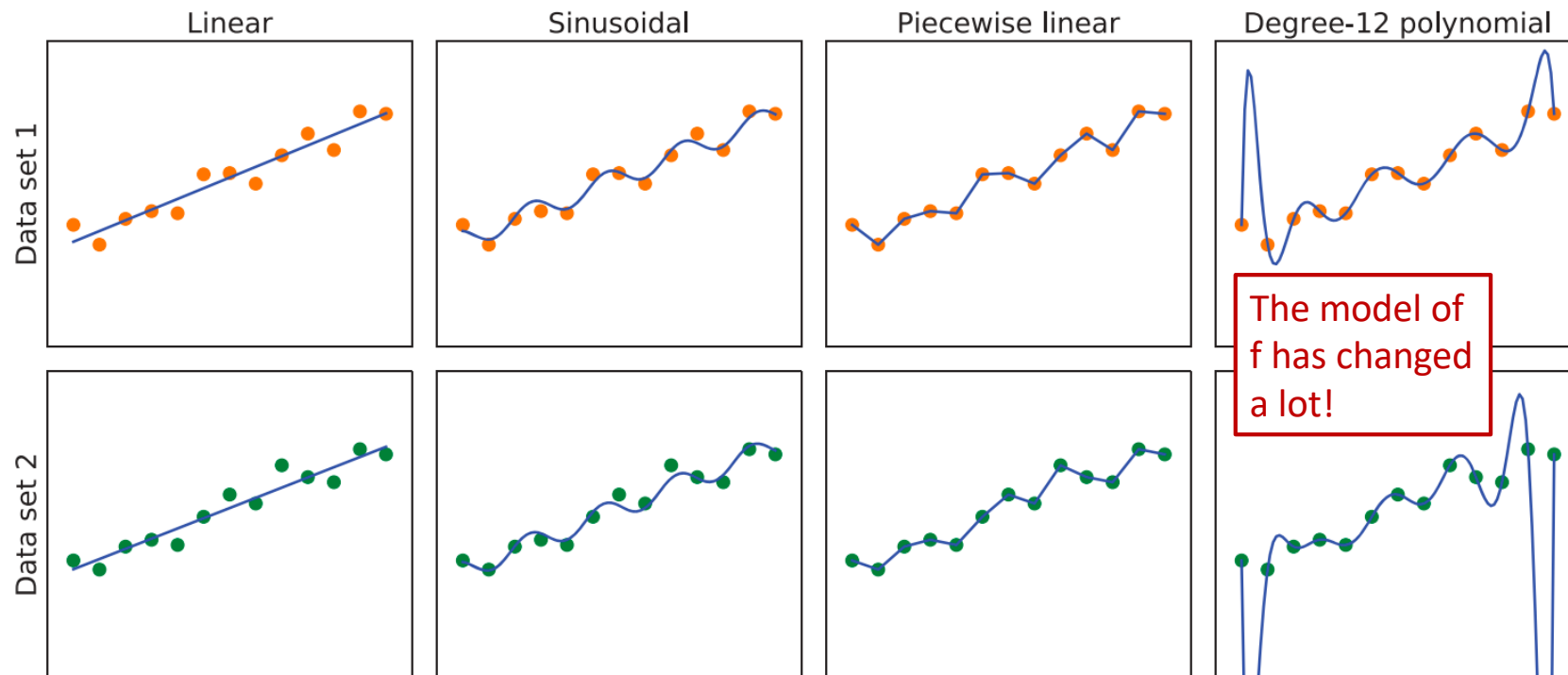
- Regression is a type of supervised learning technique that aims to predict a continuous output variable based on one or more input features.
 - While classification involves predicting discrete labels or categories, regression deals with predicting a numeric value.
 - Conceptually, regression can be seen as an extension of classification where instead of assigning an input to a predefined class, the model predicts a value on a continuous scale.



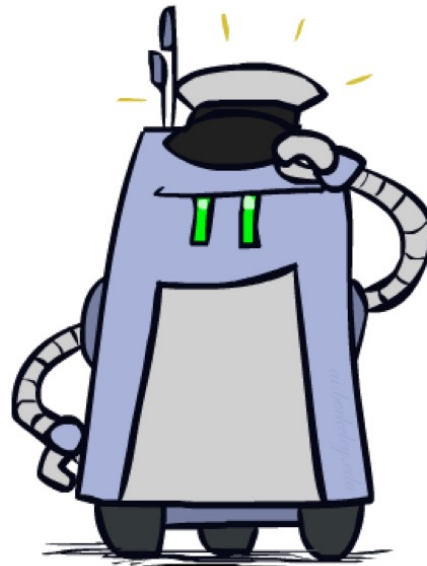
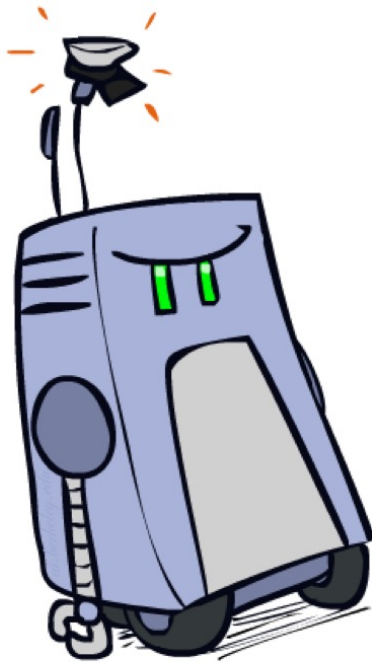
Example: Regression



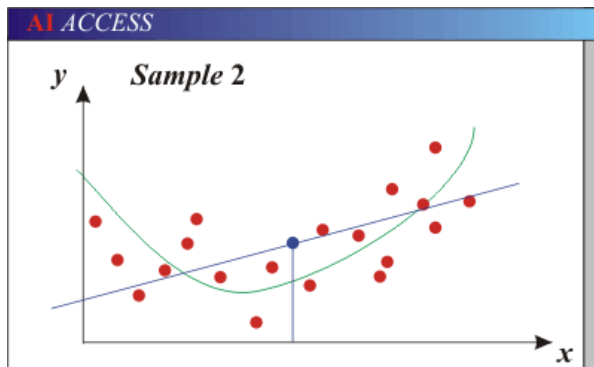
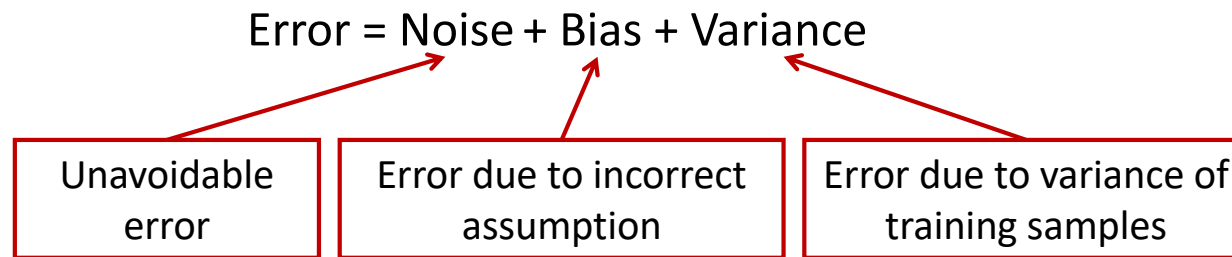
Example: Regression



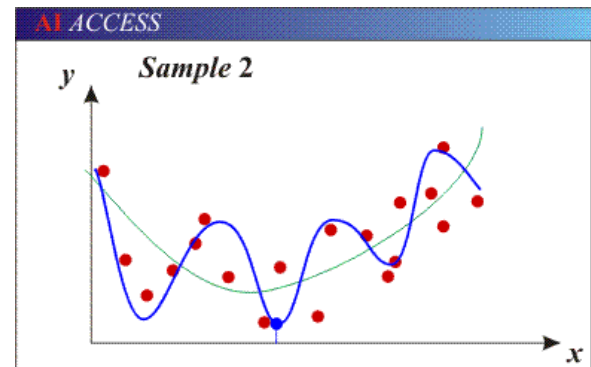
Generalization and Overfitting



Bias-Variance Trade-off

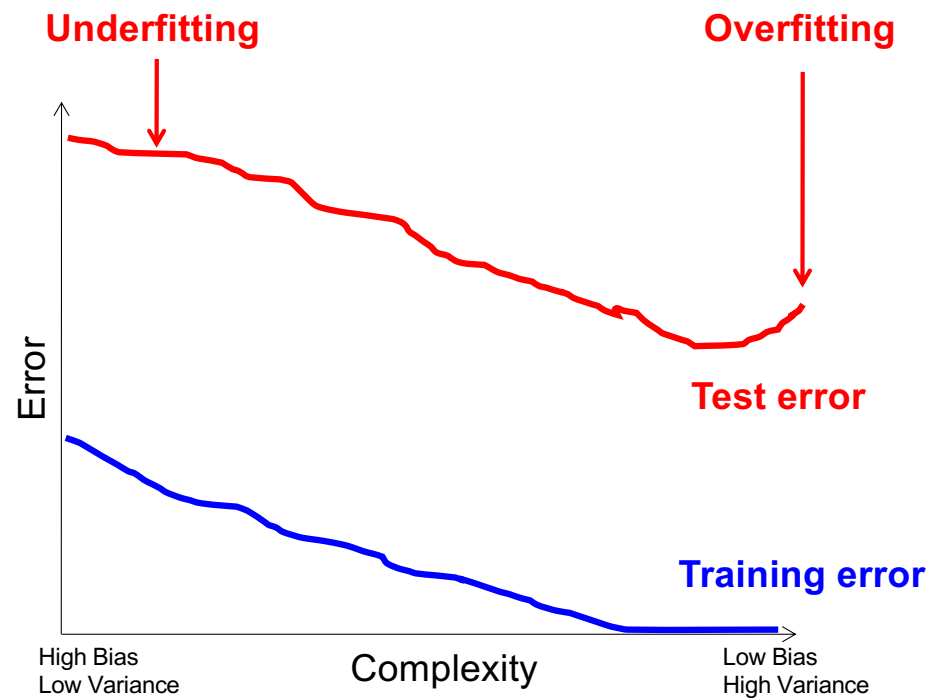


Models with too few parameters are inaccurate because of large **bias** (not enough flexibility).

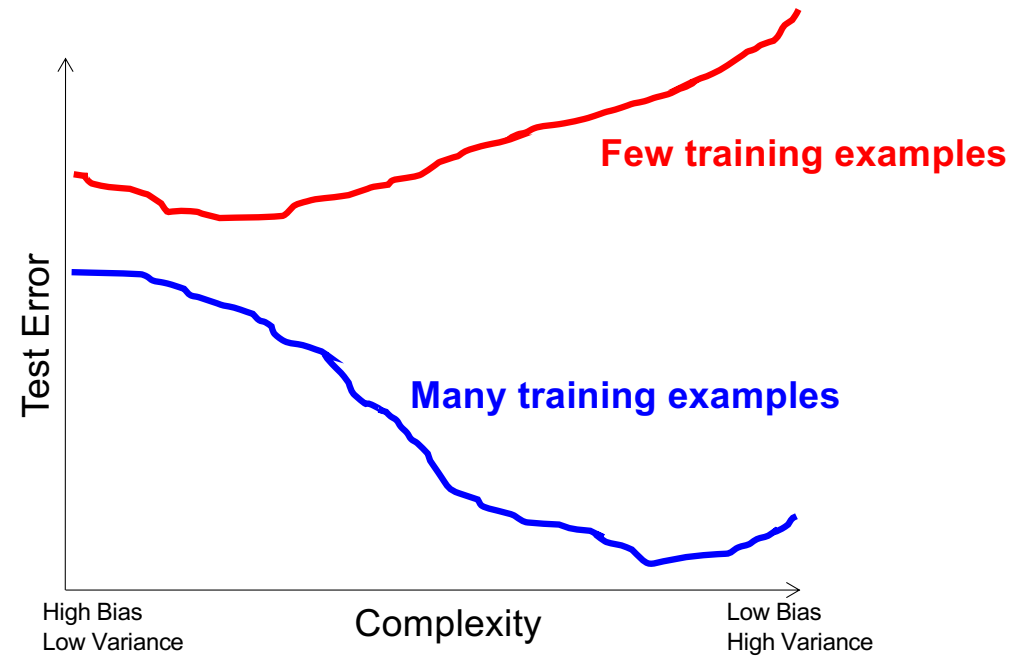


Models with too many parameters are inaccurate because of large **variance** (too sensitive to samples).

Bias-Variance Trade-off

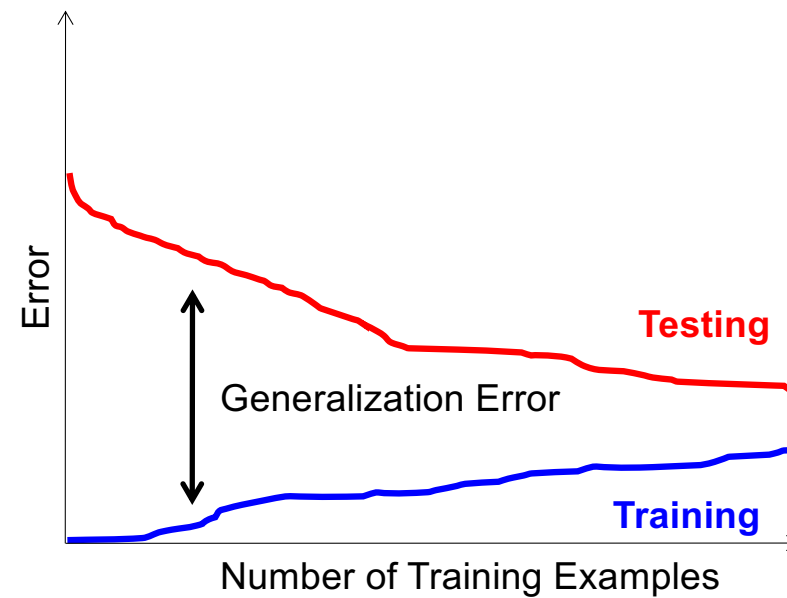


Bias-Variance Trade-off



Effect of Training Size

Fixed prediction model

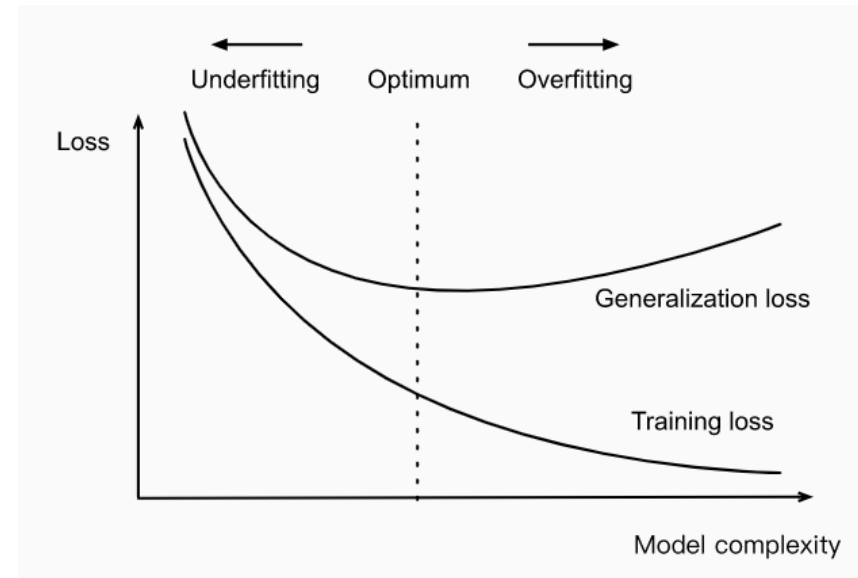


Generalization

- The goal of machine learning is to discover patterns, instead of remembering the data.
 - When a new sample comes, we can make an inference based on the pattern of the new sample.
- How to discover patterns that *generalize* — is the fundamental problem of machine learning.
- Error:
 - The *training error* is the error of our model as calculated on the training dataset;
 - We estimate the *generalization error (or test error)* by applying our model to an independent test set (e.g., constituted of a random selection of data examples that were withheld from our training set).

Generalization and Overfitting

- When we have simple models and abundant data, we expect the generalization error to resemble the training error.
- When we work with more complex models and fewer examples, we expect the training error to go down but the generalization gap to grow.

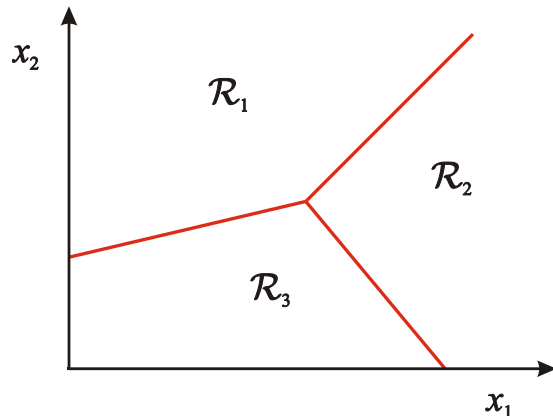


Types of Learning

	<i>Supervised Learning</i>	<i>Unsupervised Learning</i>
<i>Discrete</i>	classification or categorization	clustering
<i>Continuous</i>	regression	dimensionality reduction

Classification with Nearest-Neighbor

- Problem: assign an input sample to one of two (or more) classes
 - Assume a sample can be represented by a vector of features (e.g., x_1 and x_2);
 - All samples can then be visualized by drawing an x_1 - x_2 plane, which is the sample space;
 - Any decision rule divides the space into decision regions separated by **decision boundaries**.

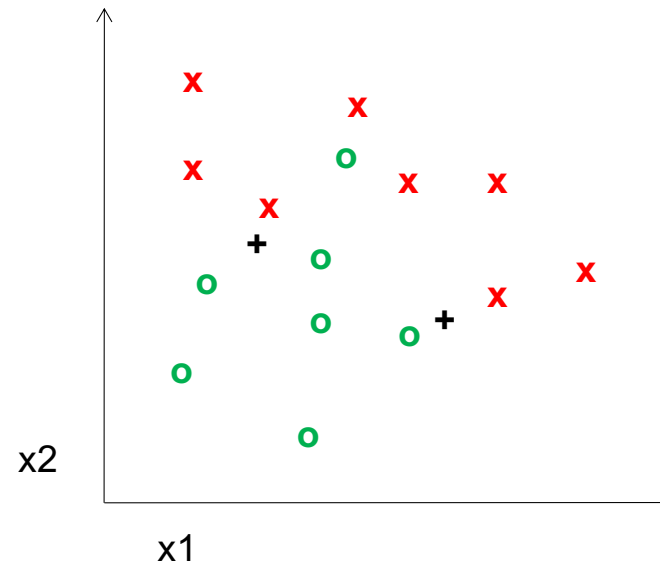


Q: Where are those red boundaries coming from?

A: One may derive them following the **nearest-neighbor rule**.

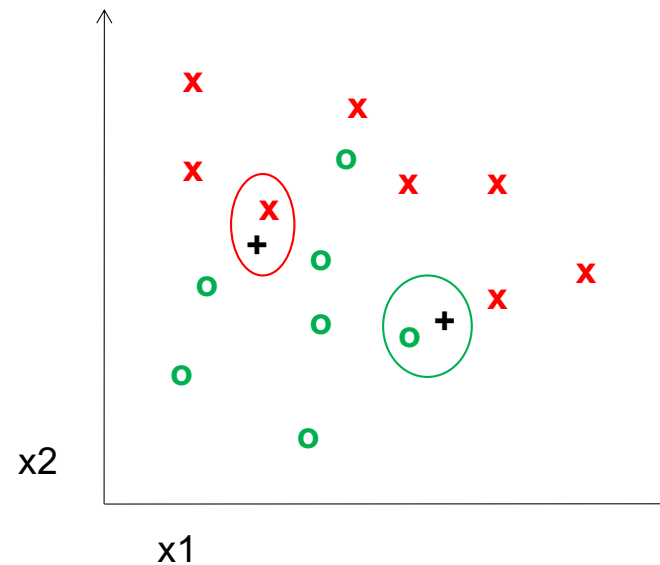
K-nearest Neighbor

- K-NN:
 - Simple, a good one to try first;
 - Instance-based learning.



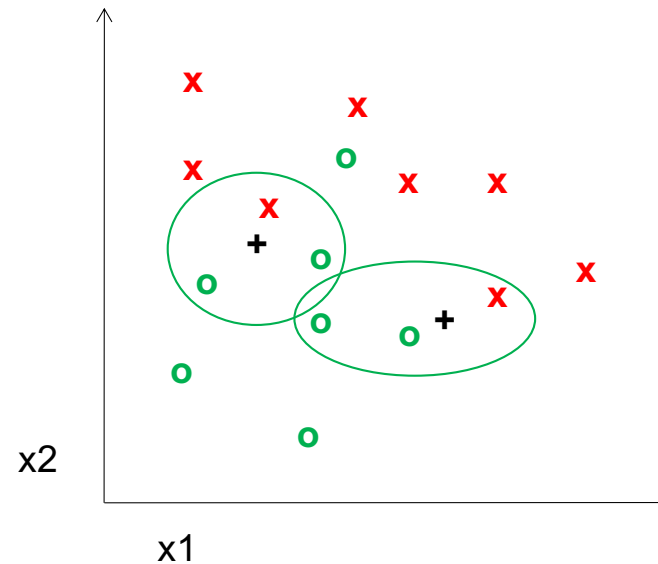
K-nearest Neighbor (K=1)

- K-NN:
 - Simple, a good one to try first;
 - Instance-based learning.



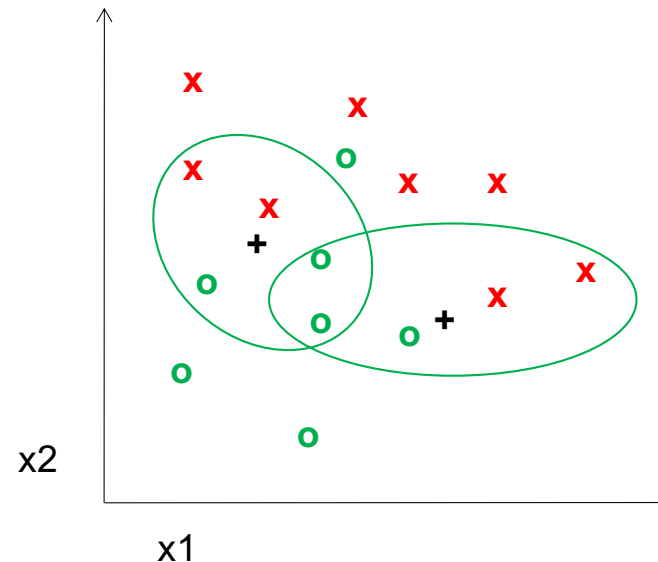
K-nearest Neighbor (K=3)

- K-NN:
 - Simple, a good one to try first;
 - Instance-based learning.



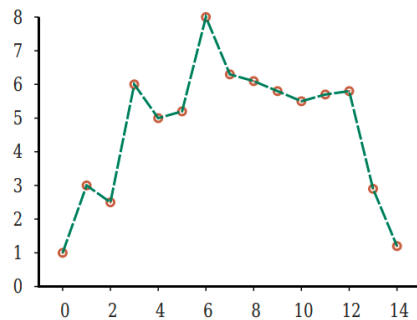
K-nearest Neighbor (K=5)

- K-NN:
 - Simple, a good one to try first;
 - Instance-based learning.
- Factors that matter:
 - Number of samples (or instances);
 - Proper K;
 - Feature representation;
 - Similarity, distance, or metric between samples.

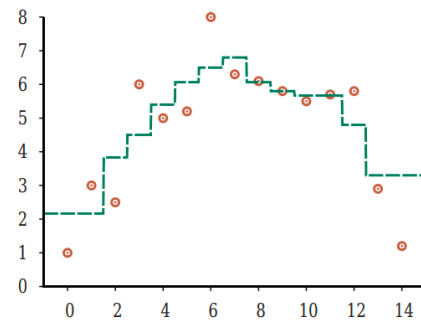


Regression with Nearest-Neighbor

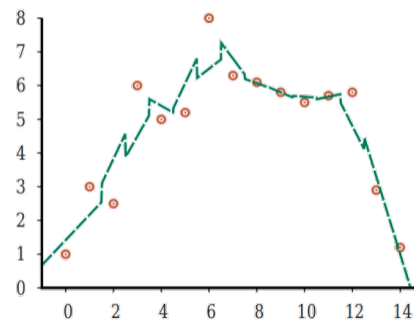
- K-nearest-neighbor rule can be easily applied to solving the regression problem, in addition to classification.
 - A standard functionality of charting in spreadsheets.



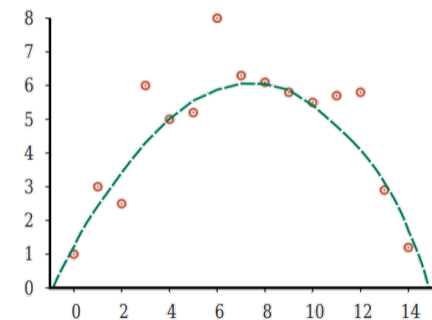
Connect the dots



3-nearest neighbors average



3-nearest-neighbors linear
regression



Locally weighted regression

Support Vector Machine

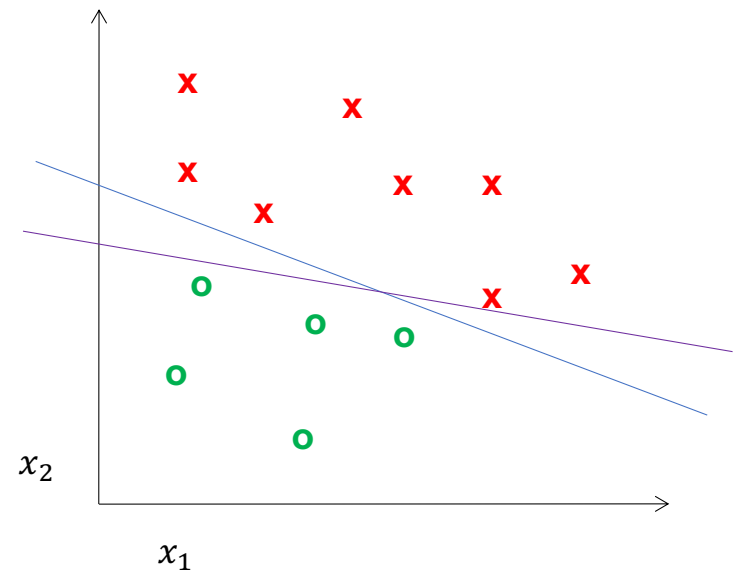
- In machine learning, **support vector machine (SVM)** is supervised learning model with associated learning algorithms that analyze data for classification and regression analysis.
 - Given a set of training examples, each marked as belonging to one of two categories, an SVM training algorithm builds a model that assigns new examples to one category or the other, making it a (non-probabilistic) **binary linear classifier**.
 - In addition to performing linear classification, SVMs can efficiently perform a **non-linear classification** using what is called the **kernel trick**, implicitly mapping their inputs into high-dimensional feature spaces.



Vladimir Vapnik

Linear SVM

- Find a **linear classifier** to separate the classes.
 - A sample point is represented by a p -dimensional feature vector (e.g, $p = 2$).
 - We want to separate points with a $(p - 1)$ -dimensional hyperplane, which is called a linear classifier.
 - But there are many such hyperplanes!
 - Which one is best?

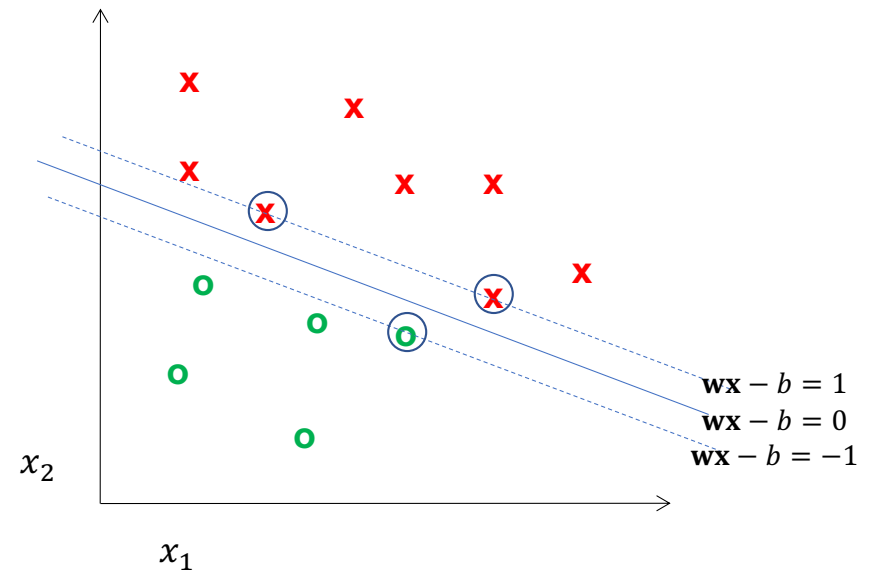


$$f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

Make a decision based on
positive/negative sign of $f(\mathbf{x})$

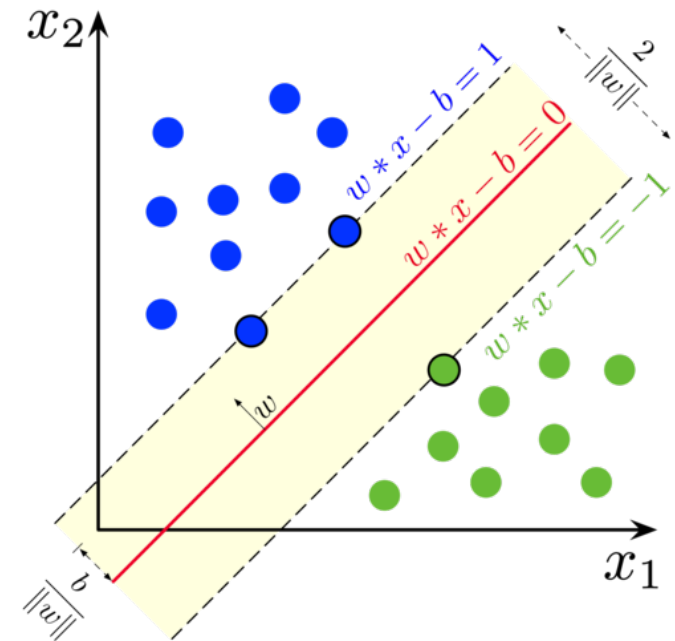
Linear SVM

- Find a **linear classifier** to separate the classes
 - SVM aims to maximize the width of the gap (margin) between the two classes.
 - If such a hyperplane exists, the distance from it to the nearest data point on each side is maximized.
 - It is known as the **maximum-margin hyperplane**, and the linear classifier it defines is known as a maximum-margin classifier.
 - The samples nearest to the maximum-margin hyperplane are called **support vectors**.



Linearly Separable Case

- Training: given n sample points
 - $\mathbf{x}_i$ is the feature vector;
 - y_i is either '+1' or '-1' indicating the class label;
 - A hyperplane can be written as $\mathbf{w}\mathbf{x} - b = 0$ for all points;
 - In a linearly separable case, we can define '+1' and '-1' hyperplanes accordingly.



Linearly Separable Case

- Training: given n sample points
 - The margin between the two ('+1' / '-1') hyperplanes is $2/\|\mathbf{w}\|$.
 - So we now have an optimization problem:

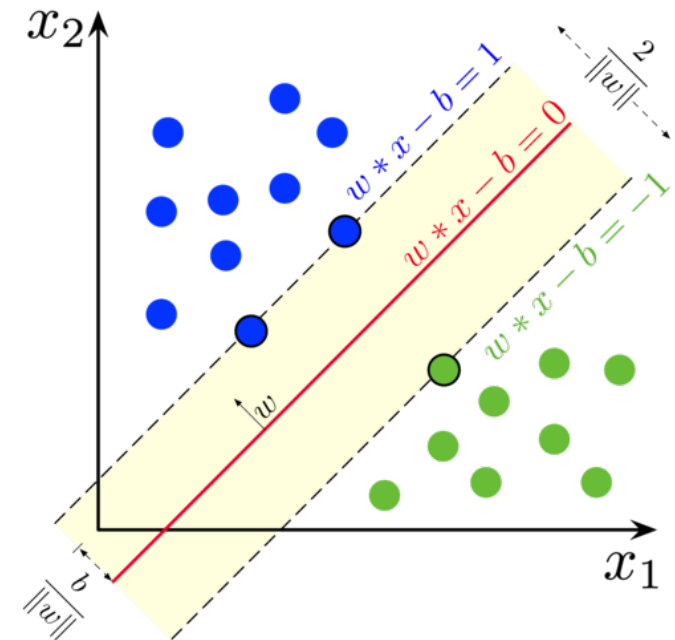
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

Maximize
margin

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$$

Classify training
data correctly

- Testing:
 - Use $f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$



From Hard-margin to Soft-margin

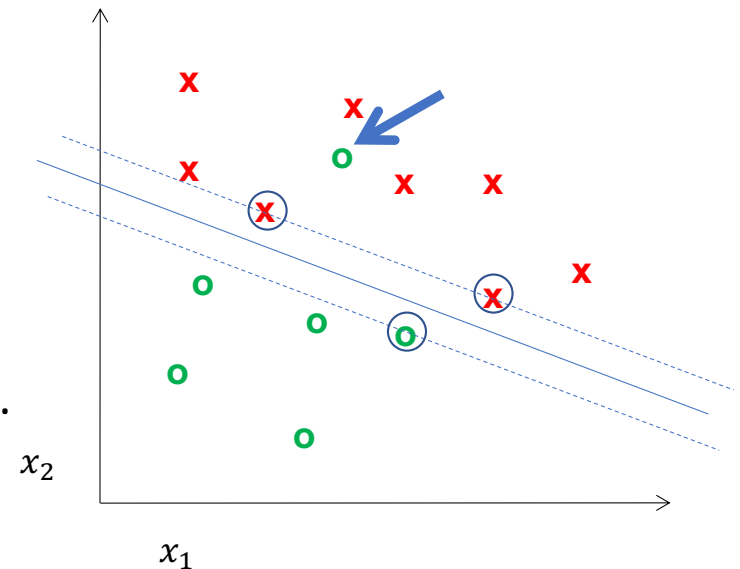
- The above linearly separable case leads to **hard margin**.
 - Sometimes, the sample points are not linearly separable.
- Need to extend SVM to have **soft-margin**:

$$\min_{\mathbf{w}, b, \zeta} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \zeta_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1 - \zeta_i$$

$$\zeta_i \geq 0$$

- ζ_i : slack variables that allow for margin violations.
- C : regularization parameter that controls the trade-off between maximizing the margin and minimizing the classification error.



From Hard-margin to Soft-margin

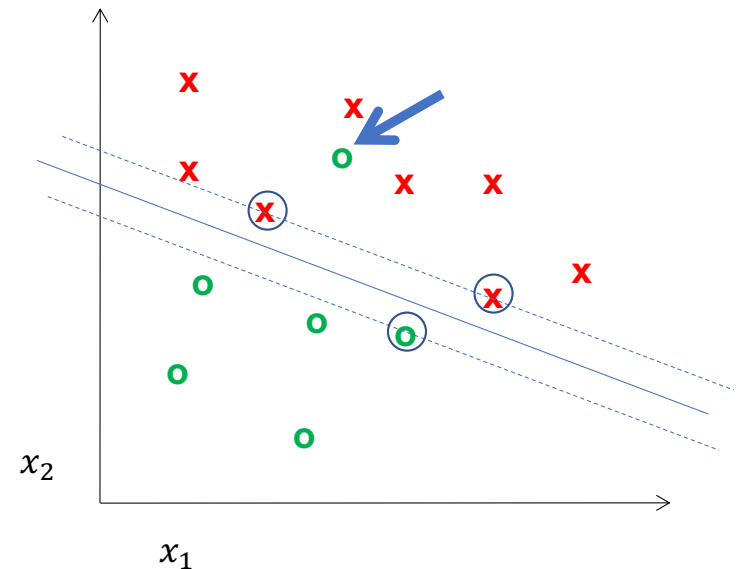
- The above linearly separable case leads to **hard margin**.
 - Sometimes, the sample points are not linearly separable.
- Need to extent SVM to have **soft-margin**:

$$\min_{\mathbf{w}, b, \zeta} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \zeta_i$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1 - \zeta_i$$

$$\zeta_i \geq 0$$

- For large value of C , it will behave similar to hard-margin SVM.
- Alternative notation: $\min_{\mathbf{w}, b, \zeta} \lambda \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \zeta_i$



How to Solve SVM?

- This is the **primal** problem for SVM:

$$\begin{aligned} \min_{\mathbf{w}, b, \zeta} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \zeta_i \\ \text{subject to} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1 - \zeta_i \\ & \zeta_i \geq 0 \end{aligned}$$

- To optimize it, we usually use **Lagrangian multiplier** and convert the primal problem to its **dual** problem.

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \\ \text{subject to} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C \end{aligned}$$

Read additional tutorial document!

How to Solve SVM?

- Why do we need a dual problem?

- Computation Efficiency: In the dual problem, the number of variables corresponds to the **number of training data points**, rather than the number of features plus one. For datasets with many features but fewer training examples, the dual problem can be computationally more efficient.
- Sparse Solutions: The solution to the dual problem often results in sparse representations, where many of the Lagrange multipliers are zero. This means that many of the training points do not contribute to defining the decision boundary, leading to the concept of **support vectors**.

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

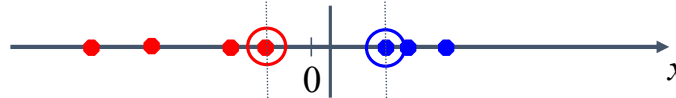
$$b = \mathbf{w} \cdot \mathbf{x}_i - y_i, \text{ for } y_i(\mathbf{w} \cdot \mathbf{x}_i - b) = 1$$

- $\alpha_i = 0$ when $\mathbf{x}_i$ lies on the correct side of the margin
 $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) > 1$
- $0 < \alpha_i < C$ when $\mathbf{x}_i$ are support vectors
 $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \leq 1$

- **Handling Non-linearity: The dual formulation of SVM allows the use of kernel functions.**

Nonlinear SVM

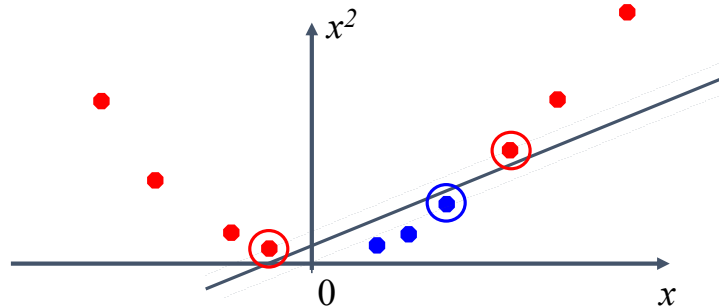
- Datasets that are linearly separable work out great:



- But what if the dataset is just too hard?

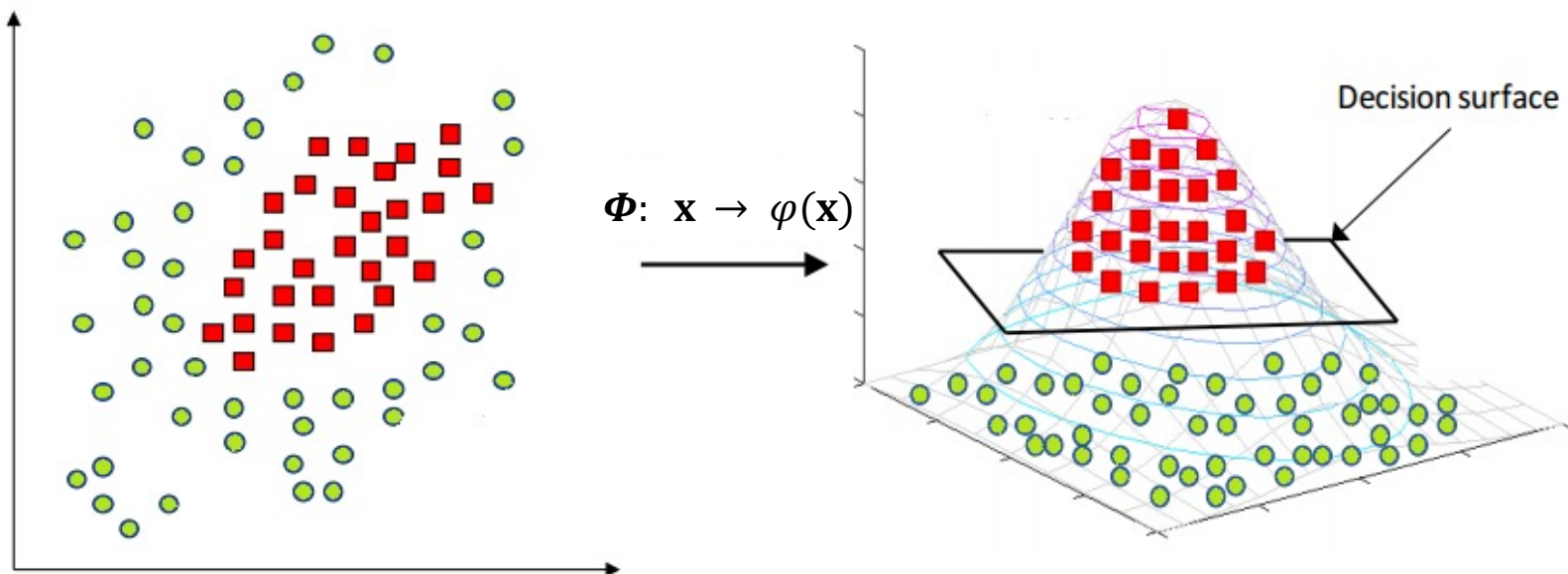


- We can map it to a higher-dimensional space:



Mapping to High-Dimensional Space

- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



Mapping to High-Dimensional Space

- Primal problem

$$\min_{\mathbf{w}, b, \zeta} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \zeta_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1 - \zeta_i$
 $\zeta_i \geq 0$

- Dual problem

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$

subject to $\sum_{i=1}^n \alpha_i y_i = 0$
 $0 \leq \alpha_i \leq C$

Apply $\varphi(\cdot)$

- The critical issue in training is to calculate the similarity between two samples, after they are mapped to high-dimensional space.
- The same story for testing (i.e., to compute similarity for a test sample with respect to all support vectors).

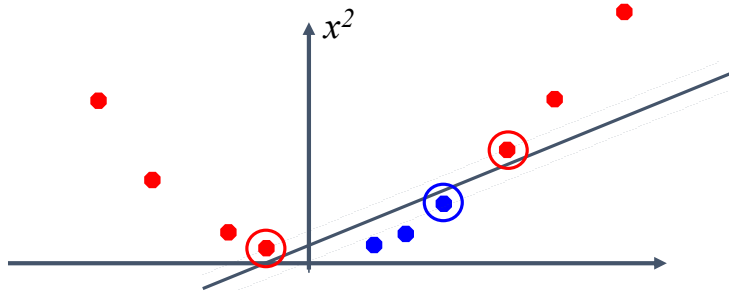
$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j))$$

subject to $\sum_{i=1}^n \alpha_i y_i = 0$
 $0 \leq \alpha_i \leq C$

Kernel trick

- **Kernel trick:** instead of explicitly computing the transformation $\varphi(\mathbf{x})$, define a kernel function k such that

$$k(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j)$$



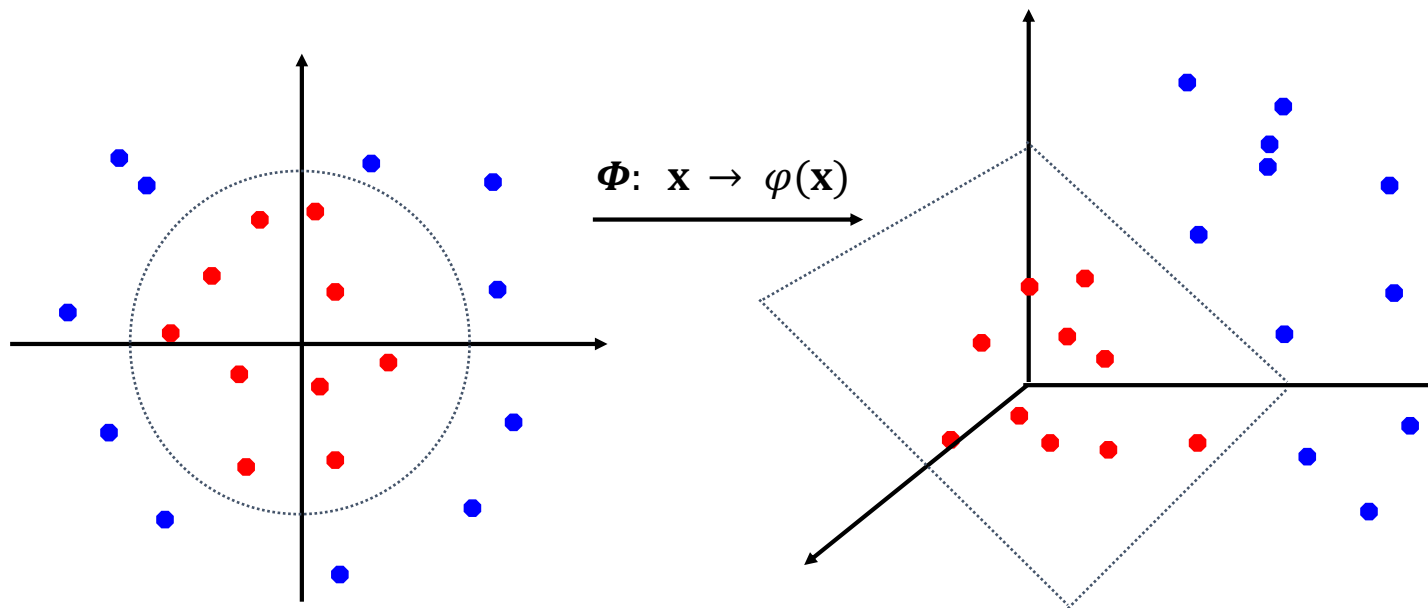
$$\varphi(x) = (x, x^2)$$

$$\varphi(x) \cdot \varphi(y) = (x, x^2) \cdot (y, y^2) = xy + x^2y^2$$

$$k(x, y) = xy + x^2y^2$$

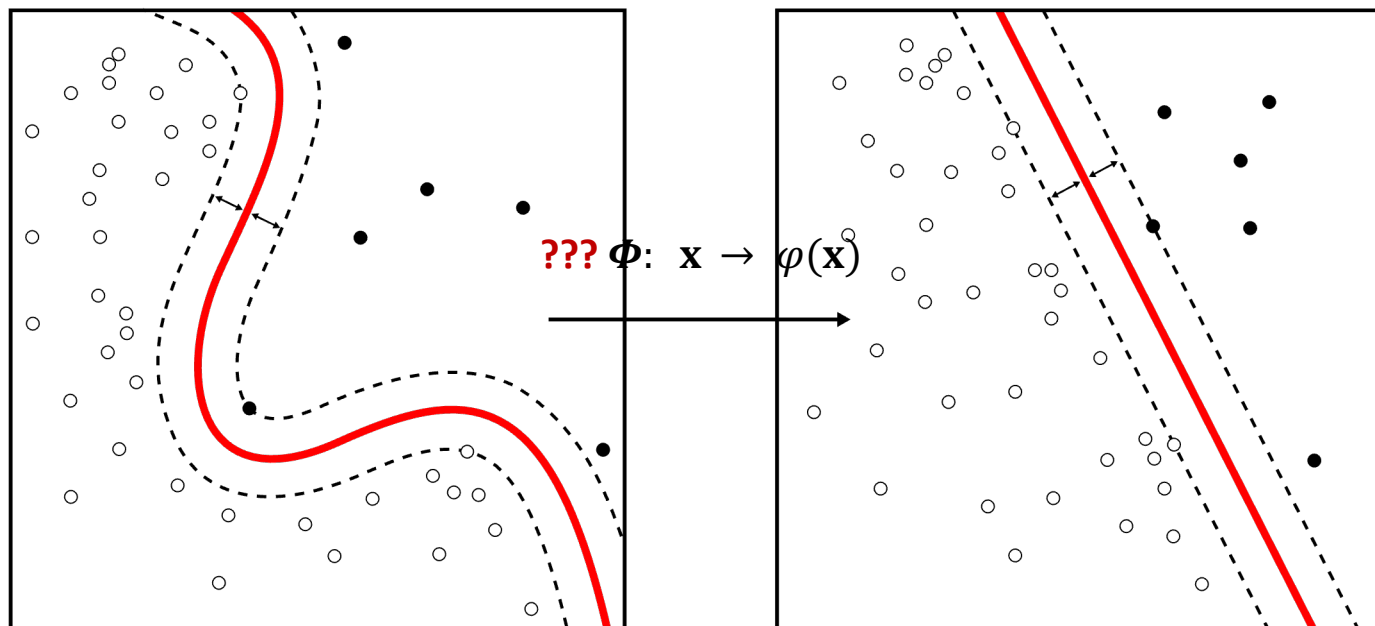
Kernel trick

- In most real case, we cannot know the mapping of $\varphi(\cdot)$ exactly.



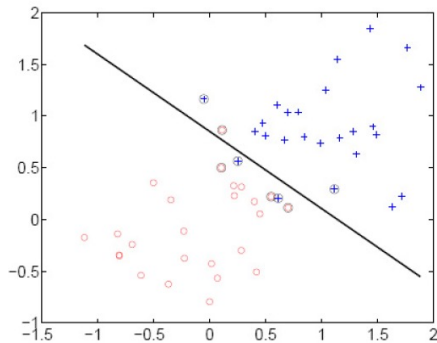
Kernel trick

- In most real case, we cannot know the mapping of $\varphi(\cdot)$ exactly.

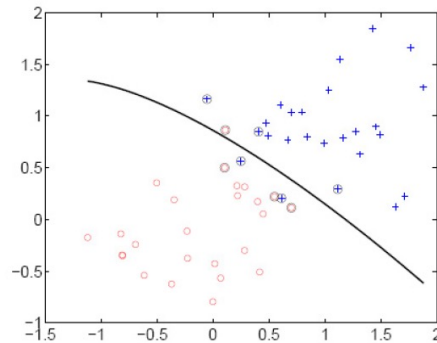


Kernel trick

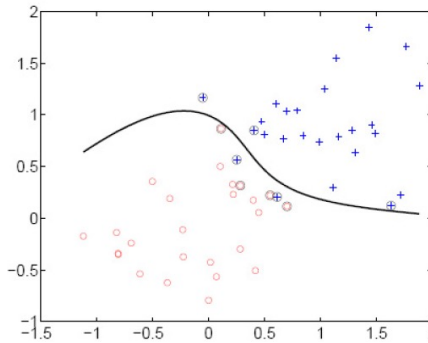
- Instead we use kernel function $k(\cdot, \cdot)$, as it is sufficient to use, e.g., by SVM.
 - Polynomial kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + c)^d$
 - c is a free parameter that scales the input space;
 - d is the degree of the polynomial.



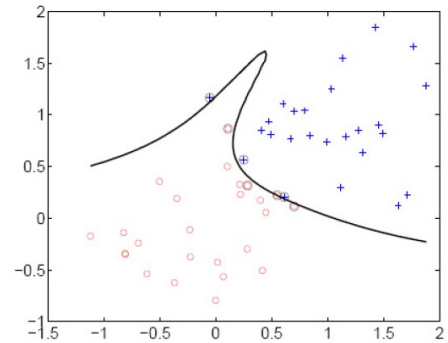
linear



2nd order polynomial



4th order polynomial



8th order polynomial

Kernel trick

- Instead we use kernel function $k(\cdot, \cdot)$, as it is sufficient to use, e.g., by SVM.
- Radial basis function (RBF) kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$
 - $\|\mathbf{x}_i - \mathbf{x}_j\|$ is the Euclidean distance between two samples.
 - σ is a free parameter that determines the width of the Gaussian function (it is related to the spread of the kernel).
 - Since the value of the RBF kernel decreases with distance and ranges between 0 (in the limit) and 1 (when $\mathbf{x}_i = \mathbf{x}_j$), it has a ready interpretation as a similarity measure.
 - General Applicability: The Gaussian kernel works well in most scenarios without prior knowledge of the data's specific characteristics. It's a universal kernel that can approximate the decision boundary of almost any shape, given enough data.

Demos

- <https://greitemann.dev/svm-demo>
- <https://scikit-learn.org/stable/modules/svm.html#classification>

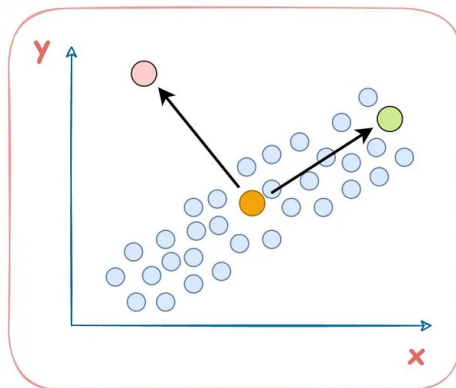
Distance Metric

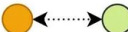
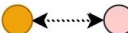
- Kernel trick (e.g., used in SVM) computes sample similarities in a high-dimensional space implicitly, by inner-product.
- Advanced distance/similarity metrics can also be used (e.g., in nearest neighbor classification) to improve the performance.

Distance Metric

- Mahalanobis Distance: Measures the distance if the data has different scales or correlated features.

$$D_M(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{(\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{S}^{-1} (\mathbf{x}_i - \mathbf{x}_j)}$$



	Euclidean Distance	Mahalanobis Distance
	4.5	1.8
	4.5	8.0
	Equal	Unequal

Distance Metric

- Cosine Similarity: Measures the cosine of the angle between two non-zero vectors, providing a measure of orientation rather than magnitude.

$$\text{CosSim}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j / (\|\mathbf{x}_i\|, \|\mathbf{x}_j\|)$$

